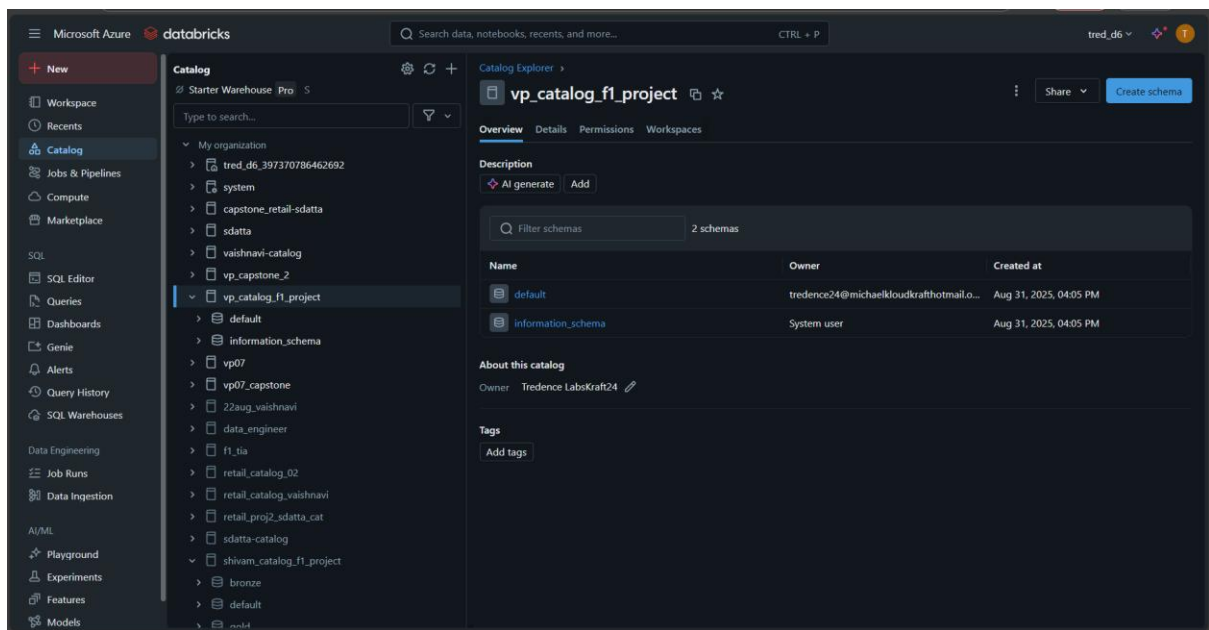
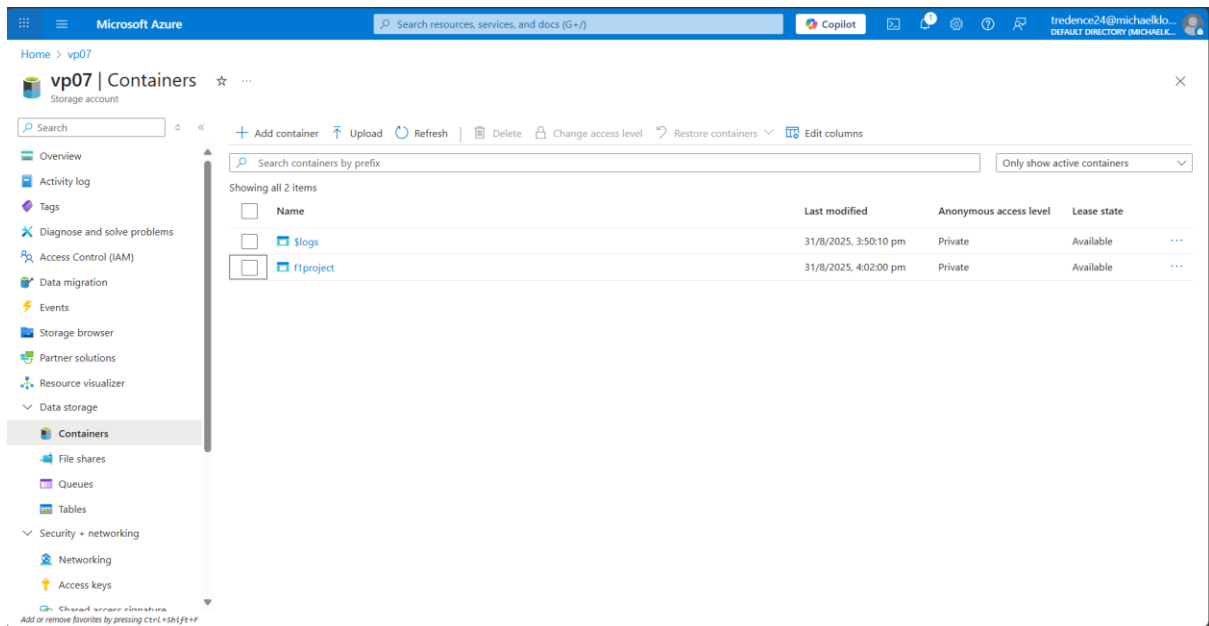
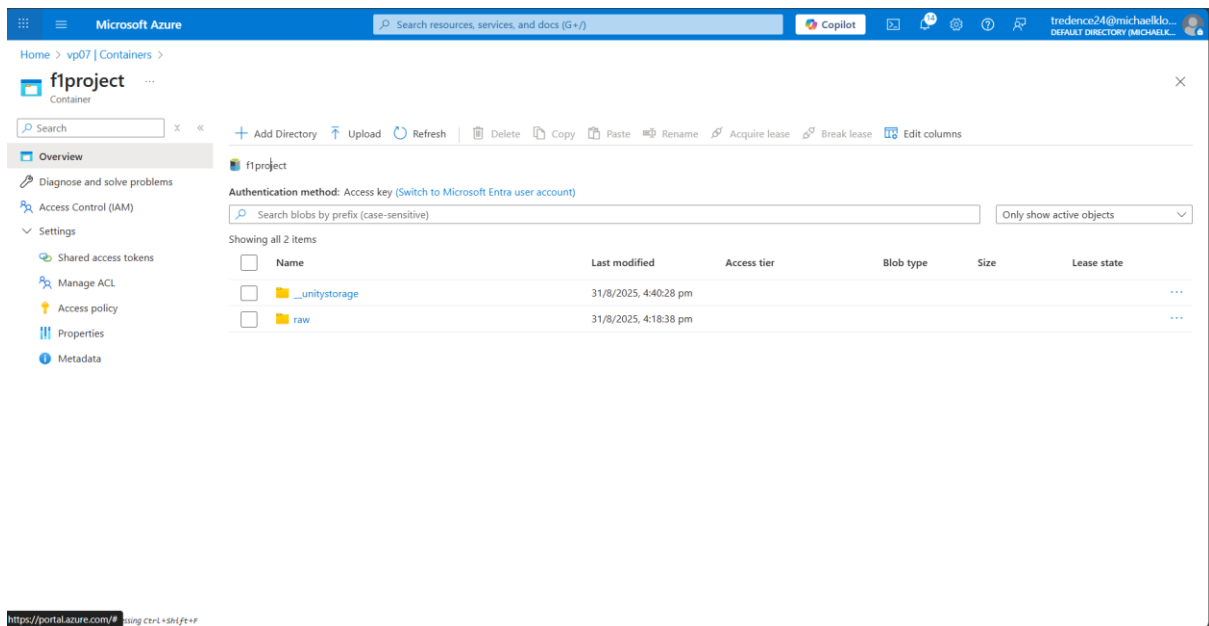
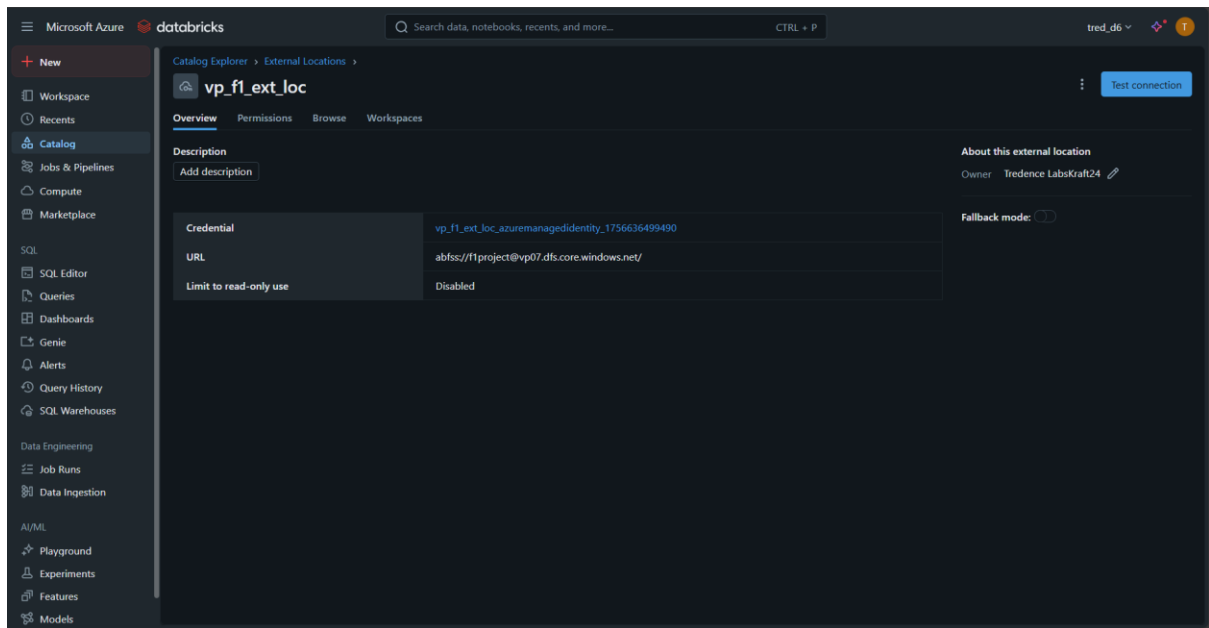


Weekend Capstone project → F1 data analysis

The screenshot shows the Microsoft Azure portal interface. At the top, there's a navigation bar with the Microsoft Azure logo, a search bar, and user information. The main content area displays the 'Overview' page for a deployment named 'vp07_1756635553959'. The deployment is marked as 'complete' with a green checkmark. Key details include: Deployment name: vp07_1756635553959, Subscription: LabsKraft, Resource group: LabsKraft2027, Start time: 8/31/2025, 3:49:22 PM, and Correlation ID: 579d481b-475f-4b73-a82a-a4a5ccb4137a. A 'Go to resource' button is visible. On the right sidebar, there are links for 'Cost Management', 'Microsoft Defender for Cloud', 'Free Microsoft tutorials', and 'Work with an expert'. The left sidebar shows navigation options: Overview, Inputs, Outputs, and Template.

The screenshot shows the Microsoft Azure portal interface for a deployment named 'AccessConnectorCreate-20250831155148'. The deployment is marked as 'complete' with a green checkmark. Key details include: Deployment name: AccessConnectorCreate-20250831155148, Subscription: LabsKraft, Resource group: LabsKraft2027, Start time: 8/31/2025, 3:52:07 PM, and Correlation ID: 526b3d90-2c1f-4d6d-a4b8-6ffa056085a1. A 'Go to resource' button is visible. The right sidebar contains the same links as the first screenshot: 'Cost management', 'Microsoft Defender for Cloud', 'Free Microsoft tutorials', and 'Work with an expert'. The left sidebar shows navigation options: Overview, Inputs, Outputs, and Template.





Microsoft Azure

Search resources, services, and docs (G+)

Copilot

fredence24@michaelk...
DEFAULT DIRECTORY (MICHAELK...

Home > vp07 | Containers >

f1project
Container

Search

+ Add Directory Upload Refresh Delete Copy Paste Rename Acquire lease Break lease Edit columns

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

Shared access tokens

Manage ACL

Access policy

Properties

Metadata

f1project > raw

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 10 items

☐	Name	Last modified	Access tier	Blob type	Size	Lease state
☐	[-]					...
☐	components.parquet	31/8/2025, 4:39:32 pm	Hot (Inferred)	Block blob	2.72 KiB	Available ...
☐	components[1].parquet	31/8/2025, 4:20:08 pm	Hot (Inferred)	Block blob	2.72 KiB	Available ...
☐	drivers.parquet	31/8/2025, 4:39:45 pm	Hot (Inferred)	Block blob	2.63 KiB	Available ...
☐	drivers[1].parquet	31/8/2025, 4:20:27 pm	Hot (Inferred)	Block blob	2.63 KiB	Available ...
☐	historical.parquet	31/8/2025, 4:39:55 pm	Hot (Inferred)	Block blob	9.64 KiB	Available ...
☐	historical[1].parquet	31/8/2025, 4:20:55 pm	Hot (Inferred)	Block blob	9.64 KiB	Available ...
☐	telemetry.parquet	31/8/2025, 4:40:05 pm	Hot (Inferred)	Block blob	214.18 KiB	Available ...
☐	telemetry[1].parquet	31/8/2025, 4:21:20 pm	Hot (Inferred)	Block blob	214.18 KiB	Available ...
☐	weather.parquet	31/8/2025, 4:40:14 pm	Hot (Inferred)	Block blob	3.01 KiB	Available ...
☐	weather[1].parquet	31/8/2025, 4:21:09 pm	Hot (Inferred)	Block blob	3.01 KiB	Available ...

Add or remove favorites by pressing Ctrl+Shift+F

Data Ingestion Code

```
# 1. Configuration
storage_account_name = "vp07"
container_name = "f1project"
catalog_name = "vp_catalog_f1_project"
schema_name = "bronze"

# List of tables to ingest for this batch run
tables_to_ingest = ["weather", "components", "components", "historical"]

# 2. Set up Catalog and Schema in Unity Catalog
spark.sql(f"USE CATALOG {catalog_name}")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {schema_name}")

print(f"Using catalog '{catalog_name}' and schema '{schema_name}'.")

# 3. Loop through and ingest each table
for table_name in tables_to_ingest:
    try:
        # Construct the source path for the Parquet file in ADLS Gen2
        source_path =
f"abfss://{container_name}@{storage_account_name}.dfs.core.windows.net/raw/{table_
name}.parquet"

        # Define the full name for the destination table in Unity Catalog
```

```

destination_table = f"{catalog_name}.{schema_name}.{table_name}"

# Read the Parquet file into a DataFrame
df = spark.read.format("parquet").load(source_path)

# --- Add ingestion metadata (Good Practice for Bronze Layer) ---
# This helps track when data was loaded and from where.
from pyspark.sql.functions import lit, current_timestamp

df_with_metadata = df.withColumn("ingestion_timestamp",
current_timestamp()) \
                        .withColumn("source_file", lit(source_path))

# Write the DataFrame to the Unity Catalog bronze table
# Using "overwrite" to ensure the job is rerunnable. Use "append" if you
expect incremental updates.
df_with_metadata.write.mode("overwrite").saveAsTable(destination_table)

print(f"✅ Successfully ingested '{table_name}.parquet' into table
'{destination_table}'.")

except Exception as e:
    print(f"❌ Failed to ingest '{table_name}'. Error: {e}")

```

```

Using catalog 'vp_catalog_f1_project' and schema 'bronze'.
✅ Successfully ingested 'weather.parquet' into table 'vp_catalog_f1_project.bronze.weather'.
✅ Successfully ingested 'components.parquet' into table 'vp_catalog_f1_project.bronze.components'.
✅ Successfully ingested 'components.parquet' into table 'vp_catalog_f1_project.bronze.components'.
✅ Successfully ingested 'historical.parquet' into table 'vp_catalog_f1_project.bronze.historical'.

```

```

from pyspark.sql.functions import col, concat_ws, to_date, when, lit,
regexp_replace
from pyspark.sql.functions import col, to_date, lower
from pyspark.sql.types import IntegerType, FloatType, DateType

# 1. Configuration
catalog_name = "vp_catalog_f1_project"
bronze_schema = "bronze"
silver_schema = "silver"

# 2. Set up Silver Schema in Unity Catalog
spark.sql(f"USE CATALOG `{catalog_name}`")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {silver_schema}")
print(f"Using catalog '{catalog_name}' and created schema '{silver_schema}'.")

```

```

from pyspark.sql.functions import col, to_date, lower

# 1. Configuration
catalog_name = "vp_catalog_f1_project"
bronze_schema = "bronze"
silver_schema = "silver"

# 2. Set the current catalog and schema
spark.sql(f"USE CATALOG {catalog_name}")
spark.sql(f"USE SCHEMA {bronze_schema}")

print(f"Reading from '{catalog_name}.{bronze_schema}.weather'")

try:
    # 3. Read the raw weather data from the bronze table
    df_weather_bronze = spark.read.table("weather")

    # 4. Apply cleaning and transformation steps
    df_weather_silver = df_weather_bronze \
        .na.drop(subset=["date", "track"]) \
        .withColumn("date", to_date(col("date"), "yyyy-MM-dd")) \
        .withColumn("weather_description", lower(col("forecast"))) \
        .withColumnRenamed("temp_c", "temperature_celsius") \
        .select("date", "track", "weather_description", "temperature_celsius")

    # 5. Write the cleaned data to the silver table
    destination_table = f"{catalog_name}.{silver_schema}.weather"
    df_weather_silver.write.mode("overwrite").saveAsTable(destination_table)

    print(f"☑ Successfully cleaned 'weather' data and saved to '{destination_table}'.")

    # Optional: Show a sample of the cleaned data
    print("\nSample of cleaned data:")
    df_weather_silver.show(5)

except Exception as e:
    print(f"❌ Failed to process 'weather' data. Error: {e}")

```

```
Reading from 'vp_catalog_f1_project.bronze.weather'
✓ Successfully cleaned 'weather' data and saved to 'vp_catalog_f1_project.silver.weather'.
```

Sample of cleaned data:

date	track	weather_description	temperature_celsius
2025-07-30	Monza	rain	26.3
2025-08-01	Silverstone	sunny	22.5
2025-08-02	Monaco	sunny	20.0
2025-08-03	Spa	rain	22.8
2025-08-04	Monza	sunny	24.1

only showing top 5 rows

```
from pyspark.sql.functions import col, lower
from pyspark.sql.types import IntegerType

# 1. Configuration
catalog_name = "vp_catalog_f1_project"
bronze_schema = "bronze"
silver_schema = "silver"

# 2. Set the current catalog
spark.sql(f"USE CATALOG {catalog_name}")

print(f"Reading from '{catalog_name}.{bronze_schema}.historical'")

try:
    # 3. Read the raw historical data from the bronze table
    df_historical_bronze = spark.read.table(f"{bronze_schema}.historical")

    # 4. Apply cleaning and transformation steps
    df_lap_data_silver = df_historical_bronze \
        .filter((col("race_id") != "INVALID_IPT") & (col("car_id") !=
"INVALID_LC1") & (col("weather") != "INVALID_NEU") & (col("tire") != "INVALID_GUS")) \
        .na.drop(subset=["lap", "lap_time", "car_id", "race_id", "pit_stop"]) \
        .withColumn("lap", col("lap").cast(IntegerType())) \
        .withColumn("pit_stop", col("pit_stop").cast(IntegerType())) \
        .withColumn("tire_compound", lower(col("tire"))) \
        .withColumn("weather_condition", lower(col("weather"))) \
        .select(
            "race_id",
            "car_id",
            "lap",
            "lap_time",
            "pit_stop",
            "tire_compound",
            "weather_condition"
        )
```

```

# 5. Write the cleaned data to the silver table
destination_table = f"{silver_schema}.lap_data"
df_lap_data_silver.write.mode("overwrite").saveAsTable(destination_table)

print(f"✅ Successfully cleaned 'historical' data and saved to
'{destination_table}'.")

# Optional: Show a sample of the cleaned data
print("\nSample of cleaned lap data:")
df_lap_data_silver.show(10)

except Exception as e:
    print(f"❌ Failed to process 'historical' data. Error: {e}")

```

```

Reading from 'vp_catalog_f1_project.bronze.historical'
✅ Successfully cleaned 'historical' data and saved to 'silver.lap_data'.

Sample of cleaned lap data:
+-----+-----+-----+-----+-----+-----+
|race_id|car_id|lap|lap_time|pit_stop|tire_compound|weather_condition|
+-----+-----+-----+-----+-----+-----+
| R001| C02| 1| 92.568| 0| hard| sunny|
| R001| C02| 2| 88.574| 0| hard| rain|
| R001| C01| 3| 87.485| 0| soft| rain|
| R001| C02| 3| 82.588| 0| hard| sunny|
| R001| C01| 4| 78.671| 0| hard| cloudy|
| R001| C02| 4| 82.728| 0| medium| rain|
| R001| C02| 5| 94.944| 0| soft| sunny|
| R001| C01| 6| 76.676| 0| hard| rain|
| R001| C02| 7| 86.946| 0| hard| cloudy|
| R001| C01| 8| 85.308| 0| medium| rain|
+-----+-----+-----+-----+-----+-----+
only showing top 10 rows

```

```

from pyspark.sql.functions import col, lower
from pyspark.sql.types import IntegerType

# 1. Configuration
catalog_name = "vp_catalog_f1_project"
bronze_schema = "bronze"
silver_schema = "silver"

# 2. Set the current catalog
spark.sql(f"USE CATALOG {catalog_name}")

print(f"Reading from '{catalog_name}.{bronze_schema}.components")

try:
    # 3. Read the raw historical data from the bronze table
    df_components_bronze = spark.read.table(f"{bronze_schema}.components")

```



```

# 4. Apply cleaning and transformation steps
df_components_silver = df_components_bronze

# 5. Write the cleaned data to the silver table
destination_table = f"{silver_schema}.components"
df_lap_data_silver.write.mode("overwrite").saveAsTable(destination_table)

print(f"✅ Successfully cleaned 'components' data and saved to
'{destination_table}'.")

# Optional: Show a sample of the cleaned data
print("\nSample of cleaned lap data:")
df_components_silver.show(10)

except Exception as e:
    print(f"❌ Failed to process 'historical' data. Error: {e}")

```

```

Reading from 'vp_catalog_f1_project.bronze.components'
✅ Successfully cleaned 'components' data and saved to 'silver.components'.

Sample of cleaned lap data:
+-----+-----+-----+-----+-----+-----+
|car_id|chassis|aero_package|engine_version|ingestion_timestamp|source_file|
+-----+-----+-----+-----+-----+-----+
|C01|SpecA|High-Downforce|EV_KNQ|2025-08-31 11:11:...|abfss://f1project...|
|C02|SpecA|Low-Downforce|EV_MIQ|2025-08-31 11:11:...|abfss://f1project...|
+-----+-----+-----+-----+-----+-----+

```

```

from pyspark.sql.functions import col, avg, min, max, sum, collect_set, lag,
row_number, first, stddev
from pyspark.sql.functions import col, avg, min, max, sum, count, collect_set,
lag, row_number
from pyspark.sql.window import Window

# 1. Configuration
catalog_name = "vp_catalog_f1_project"
silver_schema = "silver"
gold_schema = "gold"

# 2. Set up Gold Schema in Unity Catalog
spark.sql(f"USE CATALOG {catalog_name}")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {gold_schema}")

# 3. Read the silver table
lap_data_df = spark.read.table(f"{silver_schema}.lap_data")
# =====
# Gold Table 1: Race Summary for BI and Analytics

```

```

# =====
print("\n--- Creating Gold Table: race_summary ---")
try:
    race_summary_df = lap_data_df \
        .groupBy("race_id", "car_id") \
        .agg(
            count("lap").alias("total_laps"),
            min("lap_time").alias("fastest_lap_time"),
            avg("lap_time").alias("average_lap_time"),
            sum("pit_stop").alias("total_pit_stops"),
            collect_set("tire_compound").alias("tire_compounds_used")
        )

    # Write to Gold layer
    destination_table = f"{gold_schema}.race_summary"
    race_summary_df.write.mode("overwrite").saveAsTable(destination_table)
    print(f"✅ Successfully created aggregated table '{destination_table}'.")
    race_summary_df.show(5, truncate=False)

except Exception as e:
    print(f"❌ Failed to create 'race_summary' table. Error: {e}")

# =====
# Enhanced Gold Table: Feature Engineering for Pit Stop Prediction (ML)
# =====
print("\n--- Creating Enhanced Gold Table: pit_stop_features ---")
try:
    # --- Part 1: Initial feature calculation (stint_id, tire_age) ---
    window_spec_by_lap = Window.partitionBy("race_id", "car_id").orderBy("lap")

    features_df_part1 = lap_data_df \
        .withColumn("stint_id", sum("pit_stop").over(window_spec_by_lap) + 1)

    window_spec_by_stint = Window.partitionBy("race_id", "car_id",
        "stint_id").orderBy("lap")

    features_df_part1 = features_df_part1 \
        .withColumn("tire_age_laps", row_number().over(window_spec_by_stint))

    # --- Part 2: Calculate baseline and average metrics ---
    # Calculate the baseline pace (avg of first 2 laps) for each stint
    stint_baseline_df = features_df_part1 \
        .filter(col("tire_age_laps") <= 2) \
        .groupBy("race_id", "car_id", "stint_id") \
        .agg(avg("lap_time").alias("stint_baseline_lap_time"))

```

```

# Calculate the average pace for the entire race for each car
race_avg_df = features_df_part1 \
    .groupBy("race_id", "car_id") \
    .agg(avg("lap_time").alias("race_avg_lap_time"))

# --- Part 3: Join metrics back and calculate final features ---
final_features_df = features_df_part1 \
    .join(stint_baseline_df, ["race_id", "car_id", "stint_id"], "left") \
    .join(race_avg_df, ["race_id", "car_id"], "left") \
    .orderBy("race_id", "car_id", "lap") \
    .withColumn("lap_time_degradation", col("lap_time") -
col("stint_baseline_lap_time")) \
    .withColumn("lap_time_vs_race_avg", col("lap_time") -
col("race_avg_lap_time")) \
    .withColumn("lap_time_volatility",
stddev("lap_time").over(window_spec_by_lap.rowsBetween(-3, 0))) # 4-lap rolling
std dev

# --- Part 4: Final Selection of Columns for the model ---
model_input_df = final_features_df.select(
    "race_id",
    "car_id",
    "lap",
    "lap_time",
    "stint_id",
    "tire_age_laps",
    "stint_baseline_lap_time",
    "lap_time_degradation",
    "lap_time_vs_race_avg",
    "lap_time_volatility",
    "tire_compound",
    "weather_condition",
    "pit_stop" # This is the label for the ML model
).na.fill(0) # Fill any initial nulls (e.g., for volatility on lap 1) with 0

# Write to Gold layer
destination_table = f"{gold_schema}.pit_stop_features"
model_input_df.write.mode("overwrite").saveAsTable(destination_table)
print(f"✅ Successfully created enhanced feature table
'{destination_table}'.")
model_input_df.show(10)

except Exception as e:
    print(f"❌ Failed to create 'pit_stop_features' table. Error: {e}")

```

```

--- Creating Gold Table: race_summary ---
✓ Successfully created aggregated table 'gold.race_summary'.
+-----+-----+-----+-----+-----+-----+
|race_id|car_id|total_laps|fastest_lap_time|average_lap_time|total_pit_stops|tire_compounds_used|
+-----+-----+-----+-----+-----+-----+
|R005   |C02   |45        |75.593          |120.68911340996172|1          |[[hard, medium, soft]]|
|R004   |C01   |43        |75.251          |87.29693023255817 |0          |[[hard, soft, medium]]|
|R004   |C02   |42        |75.691          |84.6977857142857  |0          |[[hard, medium, soft]]|
|R005   |C01   |38        |78.709          |128.78852903811253|0          |[[hard, soft, medium]]|
|R002   |C01   |46        |77.454          |120.29858920539733|0          |[[hard, soft, medium]]|
+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

```

```

--- Creating Enhanced Gold Table: pit_stop_features ---
✓ Successfully created enhanced feature table 'gold.pit_stop_features'.
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|race_id|car_id|lap|lap_time|stint_id|tire_age_laps|stint_baseline_lap_time|lap_time_degradation|lap_time_vs_race_avg|lap_time_volatility|tire_compound|weather_condition|pit_stop|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|R001   |C01   |3| 87.485|1|1|83.078|4.4069999999999965|-36.44657389162562|0.0|soft|rain|0|
|R001   |C01   |4| 78.671|1|2|83.078|-4.4069999999999965|-45.26057389162561|6.2324391693782255|hard|cloudy|0|
|R001   |C01   |6| 76.676|1|3|83.078|-6.402000000000001|-47.255573891625616|5.751827274875348|hard|rain|0|
|R001   |C01   |8| 85.308|1|4|83.078|2.2300000000000004|-38.62357389162561|5.178494182675114|medium|rain|0|
|R001   |C01   |10| 89.68|1|5|83.078|6.602000000000004|-34.25157389162561|5.999706235864111|

```

```

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# 1. Configuration
catalog_name = "vp_catalog_f1_project"
gold_schema = "gold"
table_name = "race_summary"

# Set a professional plot style
sns.set_theme(style="whitegrid")

# 2. Load the Gold Layer Data
try:
    # Read the summary table from Unity Catalog
    race_summary_spark_df =
spark.read.table(f"{catalog_name}.{gold_schema}.{table_name}")

    # Convert to Pandas DataFrame for visualization
    # Using .toPandas() is safe here because the summary table is small and
    aggregated
    race_summary_pd_df = race_summary_spark_df.toPandas()

    print(f"Successfully loaded data from '{table_name}'.")
    print("Generating visualizations...")

except Exception as e:

```

```

print(f"❌ Failed to read data. Error: {e}")
# Stop execution if data cannot be loaded
dbutils.notebook.exit("Failed to read source gold table.")

# = a helper function in Databricks to render plots
# 3. Create Visualizations

# --- Visualization 1: Average Lap Time Comparison ---
# This chart compares the overall pace of each car across different races.
plt.figure(figsize=(12, 6))
sns.barplot(data=race_summary_pd_df, x="race_id", y="average_lap_time",
hue="car_id", palette="coolwarm")
plt.title('Average Lap Time by Race and Car', fontsize=16, fontweight='bold')
plt.xlabel('Race ID', fontsize=12)
plt.ylabel('Average Lap Time (seconds)', fontsize=12)
plt.xticks(rotation=45)
plt.legend(title='Car ID')
plt.tight_layout()
display(plt.gcf()) # display() is the Databricks function to show plots
plt.close()

# --- Visualization 2: Pit Stop Strategy Analysis ---
# This chart shows how many pit stops each car made in each race.
plt.figure(figsize=(12, 6))
sns.barplot(data=race_summary_pd_df, x="race_id", y="total_pit_stops",
hue="car_id", palette="viridis")
plt.title('Total Pit Stops by Race and Car', fontsize=16, fontweight='bold')
plt.xlabel('Race ID', fontsize=12)
plt.ylabel('Number of Pit Stops', fontsize=12)
plt.xticks(rotation=45)
plt.legend(title='Car ID')
plt.tight_layout()
display(plt.gcf())
plt.close()

# --- Visualization 3: Fastest Lap vs. Average Lap ---
# This scatter plot helps identify consistency. Cars closer to the diagonal line
# have a smaller gap between their average pace and their peak pace.
plt.figure(figsize=(10, 8))
sns.scatterplot(data=race_summary_pd_df, x="average_lap_time",
y="fastest_lap_time", hue="car_id", s=150, alpha=0.8, palette="plasma")
plt.title('Consistency: Fastest Lap vs. Average Lap Time', fontsize=16,
fontweight='bold')
plt.xlabel('Average Lap Time (seconds)', fontsize=12)
plt.ylabel('Fastest Lap Time (seconds)', fontsize=12)

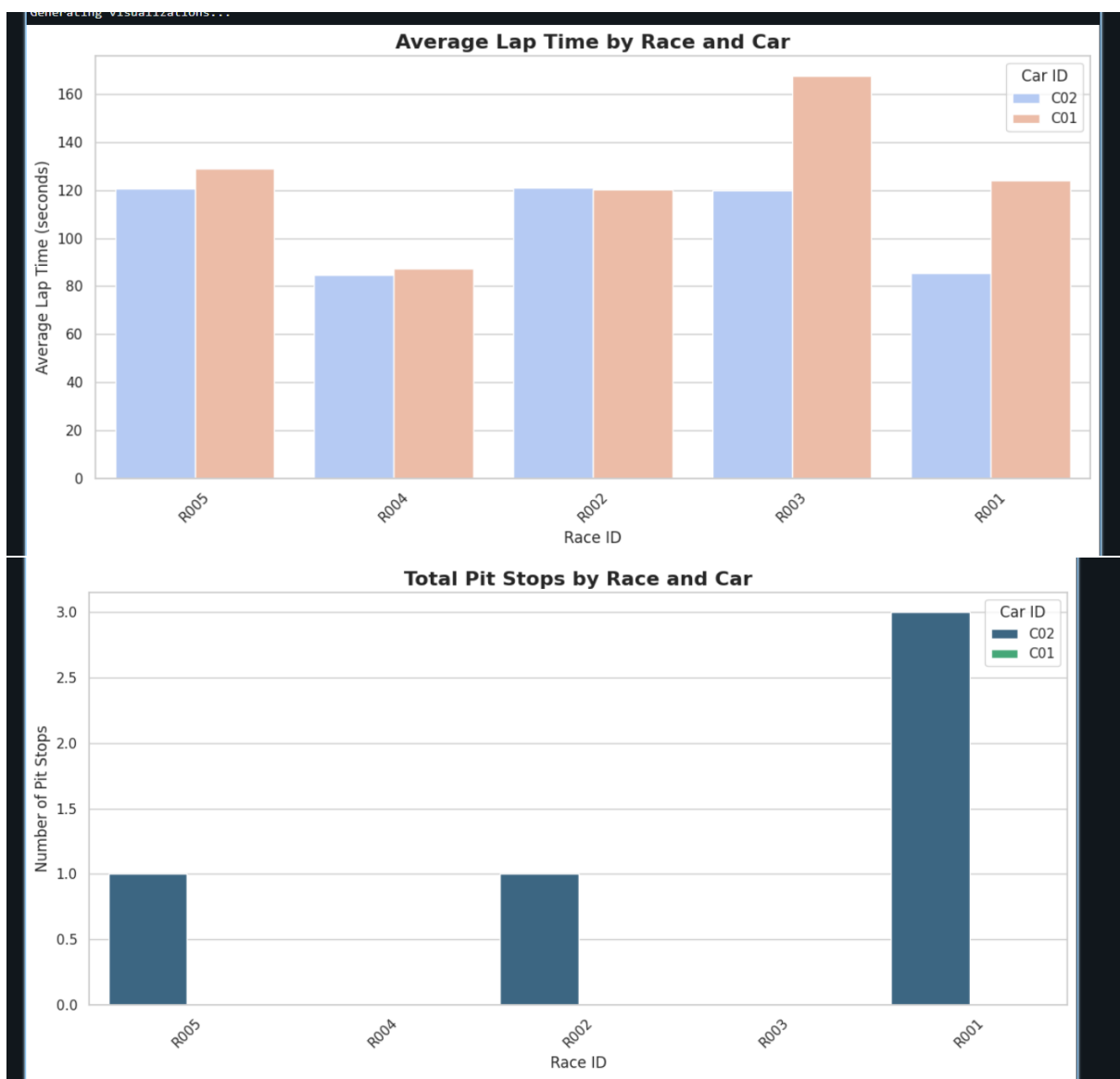
```

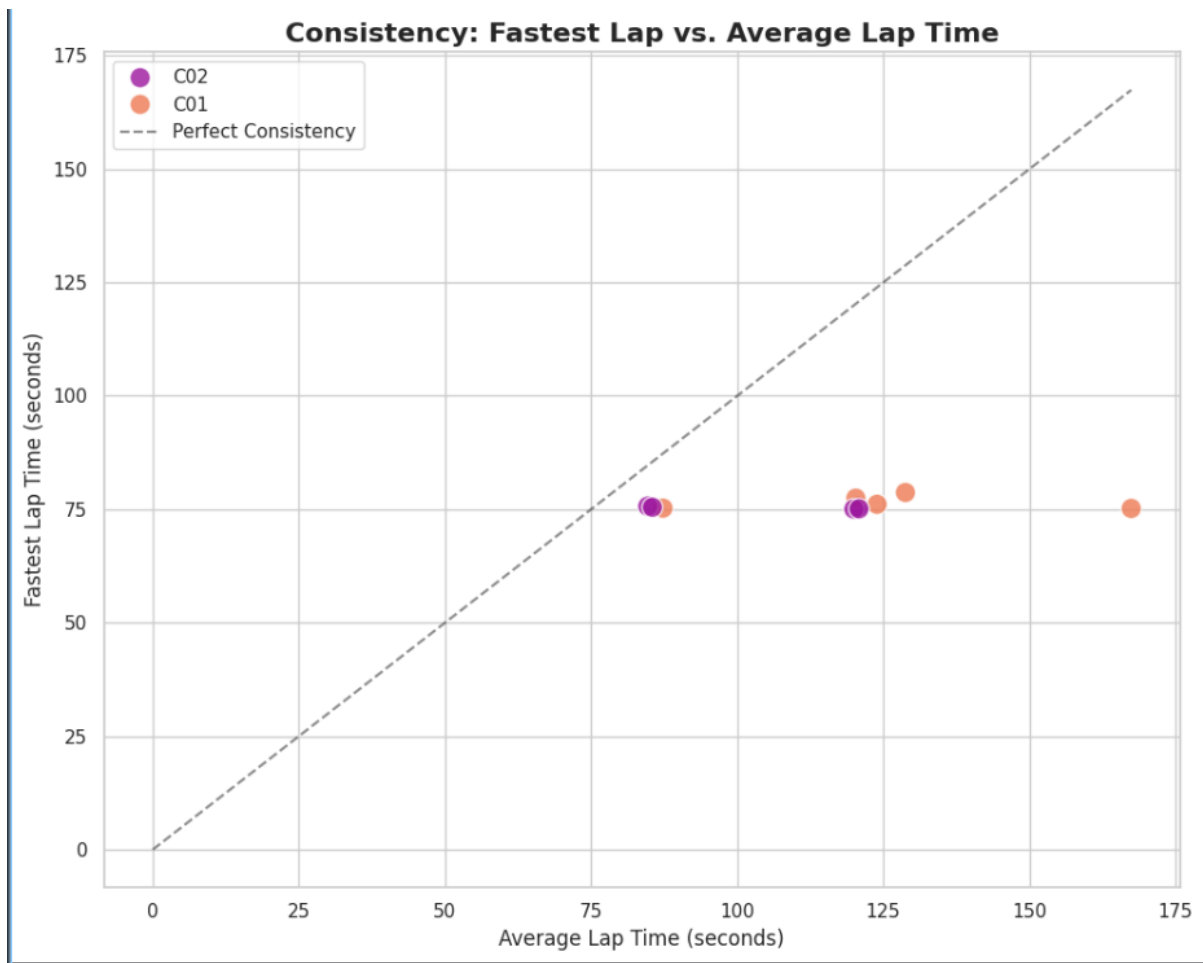
```

# Add a line for reference where x=y
max_val = race_summary_pd_df[['average_lap_time', 'fastest_lap_time']].max().max()
plt.plot([0, max_val], [0, max_val], 'k--', alpha=0.5, label='Perfect
Consistency')
plt.legend()
plt.tight_layout()
display(plt.gcf())
plt.close()

print("\nVisualizations generated successfully.")

```





ML model code→

```
import mlflow
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# 1. Load your feature data from the Gold layer
gold_df = spark.read.table("vp_catalog_f1_project.gold.pit_stop_features")
data_pd = gold_df.toPandas()

# 2. Define Features (X) and Label (y)
label = "pit_stop"
features = [
    "tire_age_laps",
    "lap_time_degradation",
```

```

        "lap_time_vs_race_avg",
        "lap_time_volatility",
        "lap_time"
    ]
X = data_pd[features]
y = data_pd[label]

# 3. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42, stratify=y)

# 4. Start an MLflow Experiment Run
with mlflow.start_run() as run:
    print("Starting MLflow Run:", run.info.run_uuid)
    mlflow.set_tag("Model", "Logistic Regression with Balanced Weights")

    # --- Train the Model with the FIX ---
    # The class_weight='balanced' parameter is the key change.
    lr = LogisticRegression(random_state=42, class_weight='balanced')
    lr.fit(X_train, y_train)

    # --- Evaluate the Model ---
    predictions = lr.predict(X_test)

    accuracy = accuracy_score(y_test, predictions)
    precision = precision_score(y_test, predictions)
    recall = recall_score(y_test, predictions)
    f1 = f1_score(y_test, predictions)

    # --- Log Metrics to MLflow ---
    mlflow.log_metric("accuracy", accuracy)
    mlflow.log_metric("precision", precision)
    mlflow.log_metric("recall", recall)
    mlflow.log_metric("f1_score", f1)

    print(f"Accuracy: {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall: {recall:.4f}")
    print(f"F1 Score: {f1:.4f}")

    # --- Log the Model to MLflow ---
    mlflow.sklearn.log_model(lr, "pit-stop-predictor")

    print("\nModel and metrics logged to MLflow successfully.")

```


▼ (1) MLflow run

Logged 1 run [🔗](#) to an [experiment](#) [🔗](#) in MLflow. [Learn more](#) [🔗](#)

- ▶  data_pd: pandas.core.frame.DataFrame = [race_id: object, car_id: object ... 11 more fields]
- ▶  gold_df: pyspark.sql.connect.dataframe.DataFrame = [race_id: string, car_id: string ... 11 more fields]
- ▶  X: pandas.core.frame.DataFrame = [tire_age_laps: int32, lap_time_degradation: float64 ... 3 more fields]
- ▶  X_test: pandas.core.frame.DataFrame = [tire_age_laps: int32, lap_time_degradation: float64 ... 3 more fields]
- ▶  X_train: pandas.core.frame.DataFrame = [tire_age_laps: int32, lap_time_degradation: float64 ... 3 more fields]

Starting MLflow Run: 8939e55a760f411eb274347509865737

Accuracy: 0.9779

Precision: 0.4000

Recall: 1.0000

F1 Score: 0.5714

2025/08/31 11:23:01 WARNING mlflow.models.model: Model logged without a signature and input example. Please set `input\_example` parameter when logging the model to auto infer the model signature.

Model and metrics logged to MLflow successfully.

★ View run carefree-pug-745 at: <https://adb-397370786462692.12.azuredatabricks.net/ml/experiments/2146099688511737/runs/8939e55a760f411eb274347509865737>

📌 View experiment at: <https://adb-397370786462692.12.azuredatabricks.net/ml/experiments/2146099688511737>